

Análisis clustering

El análisis de clustering permite explorar patrones globales de expresión génica y evaluar la similitud entre muestras o genes sin utilizar información a priori sobre las condiciones experimentales. En este trabajo se aplicaron métodos de particionamiento jerárquico y k-means sobre una matriz de genes diferencialmente expresados, con el objetivo de identificar agrupamientos coherentes entre muestras y entre sondas.

Para este tutorial, utilizaremos la matriz datos normalizados `normdata.txt` que generamos en el tutorial Análisis de expresión diferencial en R.

1. Preprocesamiento y normalización

Se utilizó la matriz de expresión normalizada generada en el tutorial de análisis de expresión diferencial. Las columnas fueron renombradas según el grupo experimental para facilitar la interpretación de los resultados.

```

outdir      <- "output"

if(!file.exists(outdir)) {
  dir.create(outdir, mode = "0755", recursive=T)
}

Data.Raw    <- read.delim("Illum_data_5000.txt")
signal      <- grep("AVG_Signal", colnames(Data.Raw)) # vector de columnas con
datos
detection   <- grep("Detection.Pval", colnames(Data.Raw))

annot       <- read.delim("MouseRef-8_annot.txt")
probe_qc    <- ifelse(annot$ProbeQuality %in% c("Bad", "No match"), "Bad probes",
  "Good probes")

design       <- read.csv("YChrom_design.csv")
print(design)

Data.Raw    <- Data.Raw[probe_qc %in% "Good probes",]
annot       <- annot[probe_qc %in% "Good probes",]

rawdata     <- as.matrix(Data.Raw[,signal])
rownames(rawdata) <- Data.Raw$PROBE_ID
colnames(rawdata) <- design$Sample_Name

library(preprocessCore)
normdata    <- normalize.quantiles(rawdata)
colnames(normdata) <- colnames(rawdata)
rownames(normdata) <- rownames(rawdata)

probe_present <- Data.Raw[,detection] < 0.04
detected_per_group <- t(apply(probe_present, 1, tapply, design$Group, sum))

present     <- apply(detected_per_group >= 2, 1, any)
normdata    <- normdata[present,]
annot       <- annot[present, ]

write.table(normdata, file.path(outdir, "normdata.txt"), sep="\t", row.names=T)

```

2. Selección de genes diferencialmente expresados

```

1) Importar matriz normdata (sondas x muestras)
mydata <- read.delim("normdata.txt", check.names = FALSE, row.names = 1)

# 2) Leer diseño y renombrar columnas por grupo
design <- read.csv("YChrom_design.csv")
colnames(mydata) <- design$Group

# 3) Leer resultados DE
res <- read.csv("DE_results.csv", check.names = FALSE)

# 4) Filtrar sondas DE por Geno/Trt/Int (FDR <= 0.19)
fdr_th <- 0.19
keep <- (res$`FDR.Geno` <= fdr_th) | (res$`FDR.Trt` <= fdr_th) | (res$`FDR.Int`
<= fdr_th)

probes_keep <- res$ProbeID[keep]

# 5) Subset a solo DE
myDE <- mydata[rownames(mydata) %in% probes_keep, , drop = FALSE]

# 6) Chequeos
cat("Sondas (normdata):", nrow(mydata), "\n")
cat("Muestras (normdata):", ncol(mydata), "\n")
cat("Sondas DE:", nrow(myDE), "\n")
cat("Muestras (DE):", ncol(myDE), "\n")
table(colnames(myDE))

```

Chequeo

```

# 6) Chequeos
cat("Sondas (normdata):", nrow(mydata), "\n")
Sondas (normdata): 2471
cat("Muestras (normdata):", ncol(mydata), "\n")
Muestras (normdata): 16
cat("Sondas DE (en normdata):", nrow(myDE), "\n")
Sondas DE (en normdata): 1437
cat("Muestras (DE):", ncol(myDE), "\n")
Muestras (DE): 16
table(colnames(myDE))

```

```

B.C  B.I  BY.C  BY.I
  4    4    4    4

```

Se parte con **2471 sondas** tras normalización y filtros de detección

Al final quedasn **1437 sondas DE** (filtradas por genotipo / tratamiento / interacción)

Se mantienen **16 muestras**, bien balanceadas: contempla 2 tratamientos sobre 2 grupos de estudio, cada uno con 4 réplicas. Así, se tiene un total de 16 muestras.

B.C B.I BY.C BY.I 4 4 4 4

3. Clustering de muestras

Se utilizó el método del codo basado en la suma de cuadrados intra-cluster (WSS).

```
# ---- Preprocesamiento para muestras ----
# Estandarizar por sonda (cada gen aporta igual)
myDE_scaled <- t(scale(t(myDE)))

# Matriz: filas = muestras
samples_mat <- t(myDE_scaled)

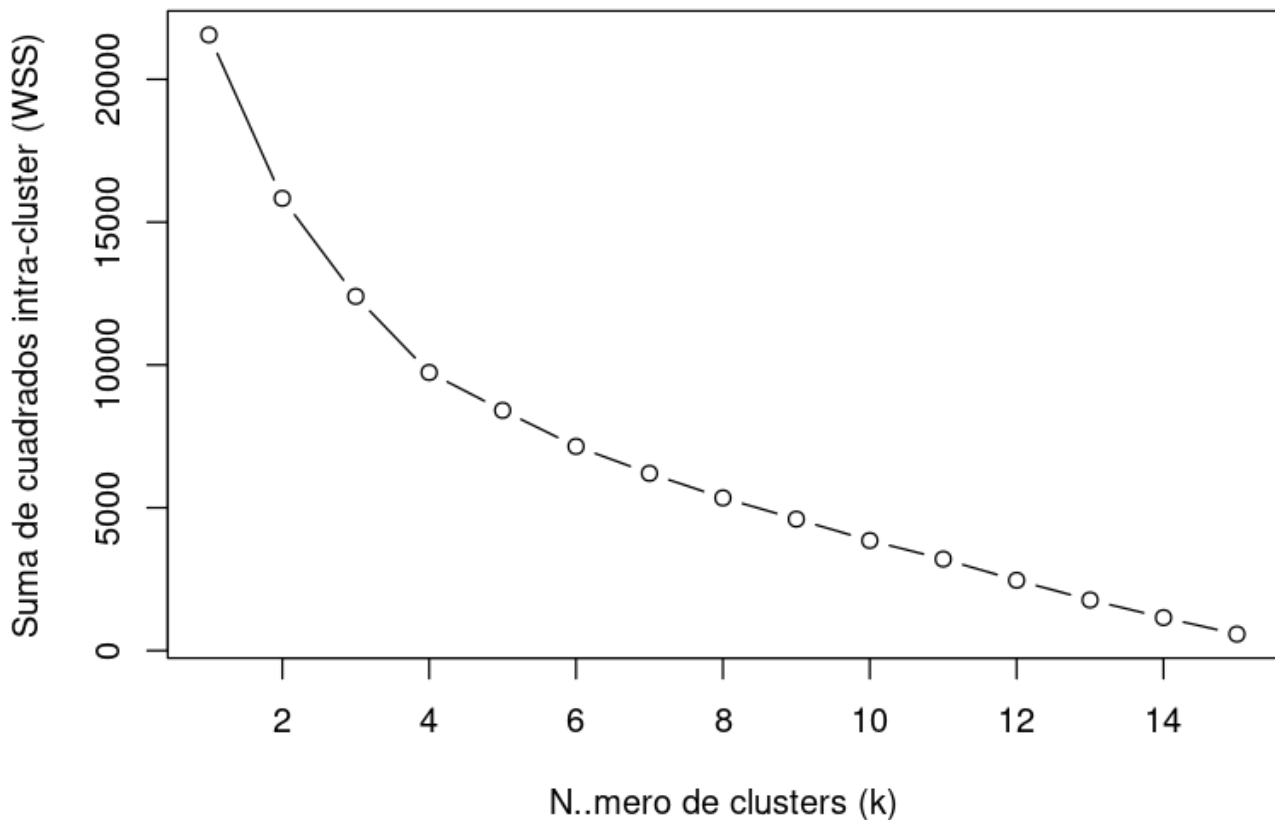
# ---- WSS ----
wss_samples <- numeric(10)
wss_samples[1] <- (nrow(samples_mat) - 1) * sum(apply(samples_mat, 2, var))

for (k in 2:15) {
  wss_samples[k] <- sum(kmeans(samples_mat, centers = k, nstart = 25)$withinss)
}

# ---- Gráfico ----
png("WSS_muestras.png", width = 800, height = 600, res = 120)

plot(1:15, wss_samples, type = "b",
     xlab = "Número de clusters (k)",
     ylab = "Suma de cuadrados intra-cluster (WSS)",
     main = "WSS para muestras")
dev.off()
```

WSS para muestras



Mirando tu gráfico, el “codo” más claro está en $k \approx 4$.

Por qué $k=4$:

- La caída de WSS es **muy fuerte de $k=1 \rightarrow 2 \rightarrow 3 \rightarrow 4$** .
- Desde $k \geq 5$ la curva sigue bajando, pero de forma **mucho más gradual** (ganancia marginal).
- Además, tú tienes **4 grupos experimentales balanceados** (B.C, B.I, BY.C, BY.I), y $k=4$ es un valor razonable e interpretable para ver si el clustering separa esos grupos (sin “forzarlo”, pero sirve como criterio práctico).

3.1 Clustering jerárquico de MUESTRAS $K=4$

```
fit <- kmeans(samples_mat2, 4)
class (fit)
# [1] "kmeans"
mydata_correcta <- t(samples_mat2)
head(mydata_correcta)
```

3.2 Calcule los promedios de expresión por cluster

```
grupo <- colnames(mydata_correcta)

tabla_grupos <- sapply(unique(grupo), function(g) {
  rowMeans(mydata_correcta[, grupo == g, drop = FALSE])
})

tabla_grupos <- as.data.frame(tabla_grupos)
head(tabla_grupos)
```

	B.I	BY.I	B.C	BY.C	B.C.1	B.I.1
ILMN_1259621	-0.19366994	-0.2155952	0.5910354	1.2390398	-0.1815038	-0.4233485
ILMN_2876302	0.54080123	1.0313277	-1.4291657	-0.4016179	0.5969539	0.5574581
ILMN_2805071	0.49100163	-0.2871629	0.3571083	0.9955006	0.1335669	-0.4587364
ILMN_2625451	0.01970249	-1.1729328	0.7774769	0.9143389	0.4049113	0.6950654
ILMN_1224651	0.98509676	0.9850968	-0.2420149	-1.6138482	-0.2981649	0.8564190
ILMN_2667100	0.09706873	-0.7232315	0.6932759	0.5607883	0.8363389	-0.3702227
	BY.I.1	BY.C.1	BY.C.2	B.C.2	BY.I.2	B.I.2
ILMN_1259621	-1.9267259	1.1542779	0.7382491	-1.2586462	1.1478396	-0.7639938
ILMN_2876302	-0.8604922	-0.2321994	-0.2586771	-1.5197639	-0.2891382	1.6846295
ILMN_2805071	-0.6296870	0.9955006	1.1511731	-0.1770191	0.3571083	-0.7029435
ILMN_2625451	-1.4149154	0.5222805	-0.2822163	0.1328621	-1.1387205	1.8378270
ILMN_1224651	0.4009387	-1.5072389	-1.2688322	0.8901896	-0.5165461	0.8564190
ILMN_2667100	-1.5869494	0.9334648	-0.4074361	2.0026258	-0.2455756	-1.1588074
	B.I.3	BY.I.3	BY.C.3	B.C.3		
ILMN_1259621	0.8788780	-1.60947887	0.26655559	0.5570867		
ILMN_2876302	0.7320117	1.56533907	-1.18998944	-0.5274775		
ILMN_2805071	-1.7657615	0.64776189	1.15117307	-2.2585839		
ILMN_2625451	0.1328621	-2.04701025	0.09618811	0.5222805		
ILMN_1224651	1.1340119	-0.04632027	-1.37090002	0.7556938		
ILMN_2667100	-0.1983558	-1.68874851	0.93346476	0.3222998		

4. Clustering de sondas

4.1 Selección del número de clusters

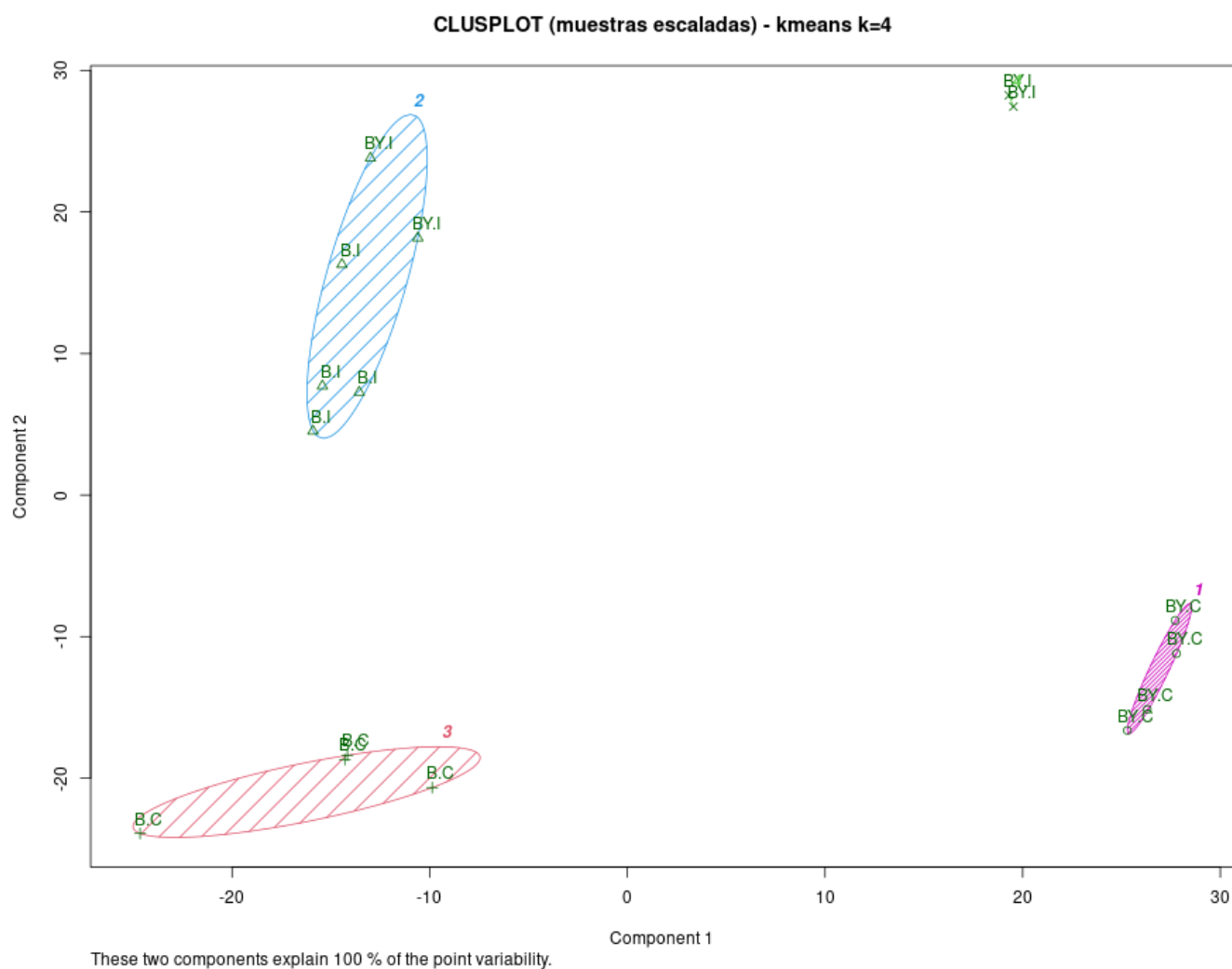
```

# Kmeans
set.seed(123)
fit <- kmeans(samples_mat, centers = 4, nstart = 25)

# PCA con prcomp (sí funciona aunque variables > muestras)
pca <- prcomp(samples_mat, center = TRUE, scale. = FALSE)
pc2 <- pca$x[, 1:2] # 16 x 2

png("Clusplot_muestra_k4.png", width = 600, height = 500)
clusplot(pc2, fit$cluster, color = TRUE, shade = TRUE,
          labels = 2, lines = 0,
          main = "CLUSPLOT (muestras escaladas) - kmeans k=4")
dev.off()

```

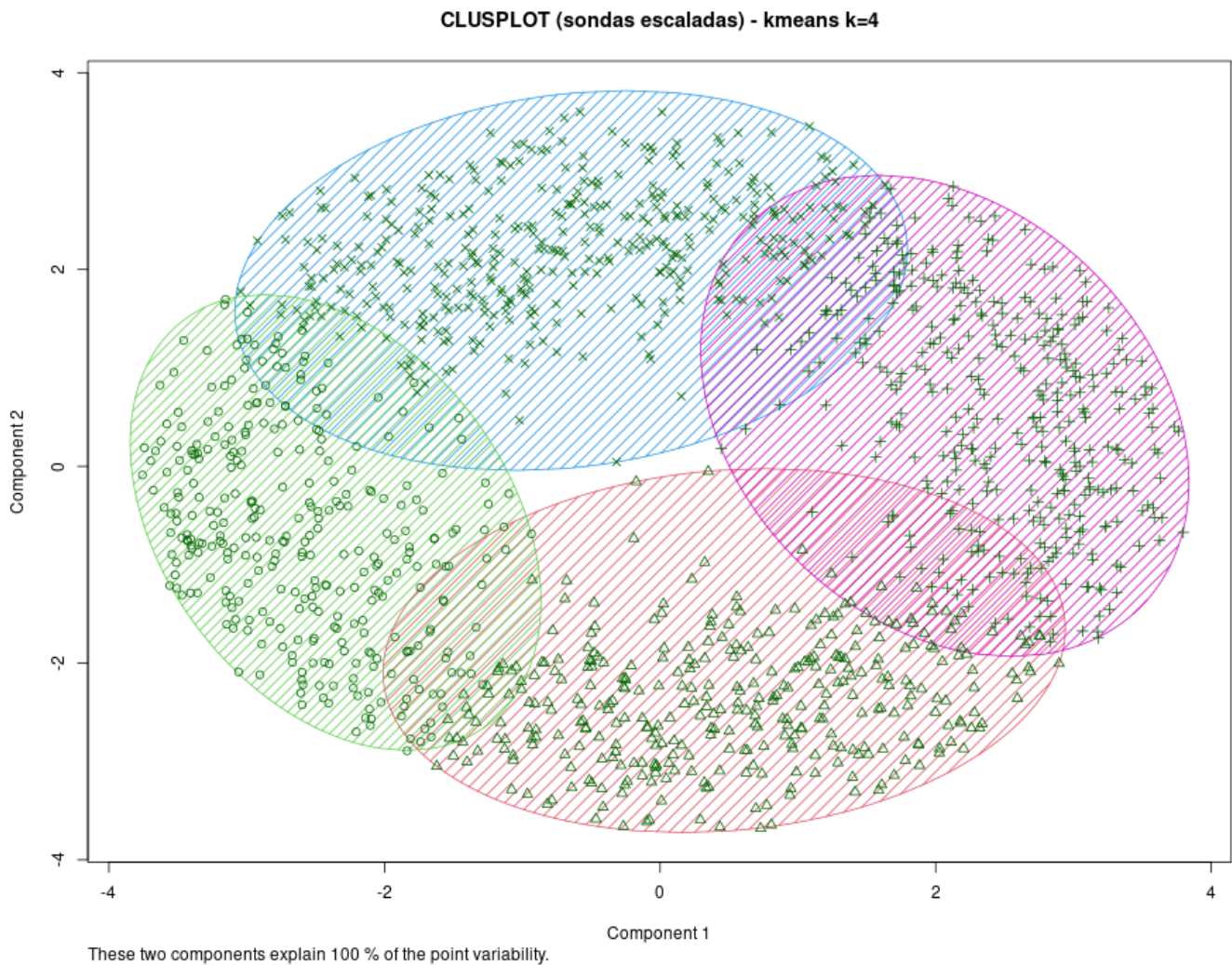


```
# kmeans por SONDA (largo = nrow(mydata_correcta) = 1437)
set.seed(123)
fit_genes <- kmeans(mydata_correcta, centers = 4, nstart = 25)

# Reducir a 2 componentes para clusplot
pca_g <- prcomp(mydata_correcta, center = TRUE, scale. = FALSE)
pc2_g <- pca_g$x[, 1:2]

png("Clusplot_sondas_k4.png", width=900, height=700)
clusplot(pc2_g, fit_genes$cluster, color=TRUE, shade=TRUE, labels=0, lines=0,
+         main="CLUSPLOT (sondas) - kmeans k=4")
dev.off()
null device
```

4.2 Clustering jerárquico de sondas



5. Clúster jerárquico

Hay una amplia gama de enfoques de agrupamiento jerárquico. He tenido buena suerte con el método de Ward que se describe a continuación.

Distancia: euclidiana

Datos: estandarizados por sonda

Método de enlace: Ward.D2

```
# ---- Distancia euclidiana entre muestras ----
d_samples <- dist(samples_mat, method = "euclidean")

# ---- Clustering jerárquico ----
hc_samples <- hclust(d_samples, method = "ward.D2")

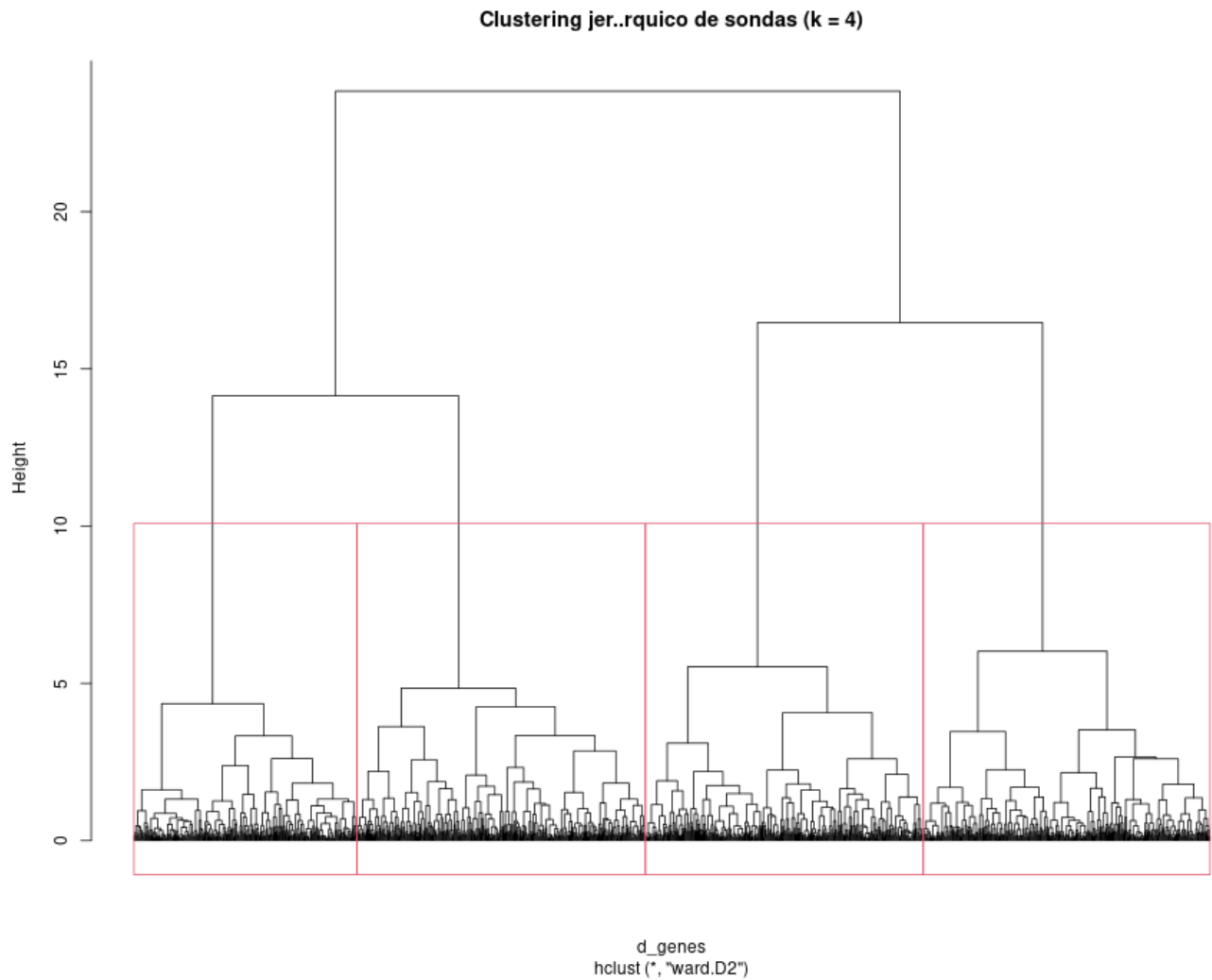
# ---- Dendrograma final con rectángulos (k = 4) ----
plot(hc_samples,
     main = "Clustering jerárquico de muestras\n(Euclidiana + Ward.D2)",
     xlab = "",
     sub = "",
     hang = -1,
     cex = 0.9)

rect.hclust(hc_samples, k = 4, border = "red")

-----
# ---- Distancia euclidiana entre muestras ----
d <- dist(t(samples_mat), method = "euclidean") # distance matrix
fit <- hclust(d, method="ward.D")
png("hclust_samples.png", width=600, height=500)
  plot(fit, hang = -1, cex = 0.8) # display dendrogram
dev.off()

groups <- cutree(fit, k=4) # cut tree into 4 clusters
groups
```

5.2 Clústering jerárquico de genes



El clustering de muestras mostró una separación coherente con los grupos experimentales, mientras que el clustering de sondas permitió identificar conjuntos de genes con patrones de expresión similares a través de las condiciones evaluadas.

6. Conclusiones

El uso combinado de métodos jerárquicos y k-means permitió explorar la estructura de los datos de expresión diferencial, destacando la existencia de agrupamientos consistentes tanto a nivel de muestras como de genes.